Computation for Analytics

DIANE WOODBRIDGE, PH.D

About Technical Diane

Ph.D. (and M.S) in Computer Science, UCLA. 2012, 2010

Senior Member of Technical Staff, Sandia National Laboratories. 2012 – 2016

Research Interest: Database management and machine learning in health IoT, remote sensing, oral history, etc.

And I also love working with students!



About Non-technical Diane

Born in the US, but raised in Seoul, Korea.

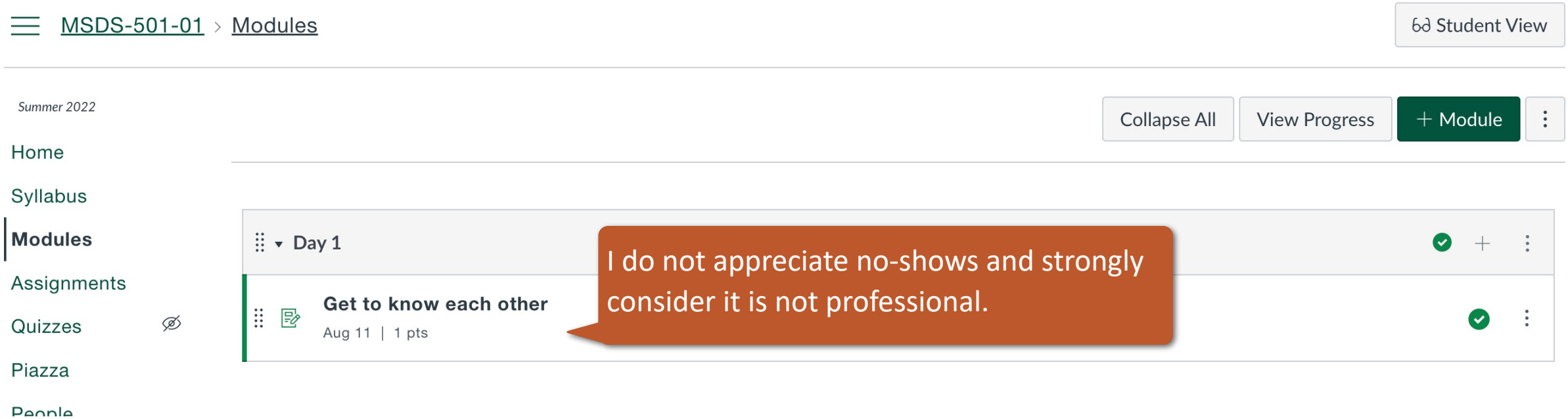
Lives in Marin County with her husband, two kids, a cat and a dog.

Loves traveling, yoga, sewing, cooking and camping.

- I recently started fishing, knitting and needle felting.



Let’s get to know each other!



About Computation for Analytics (MSDS 501)

Learning Outcomes

- Through this course, the students will learn computer organization and Python programming for data science. We will practice what a data scientist will use on a daily basis.
  - We will practice shell commands using Terminal.
  - We will use version control via Git and GitHub.
  - We will review core concepts in Python, including data types, conditional statements, loop, file I/O, data aliasing, function, and object oriented programming (OOP)
  - We will also learn concepts of packages/libraries and a few useful ones, including pytest, numpy, etc.
  - Most importantly, we will learn about how we can approach programming as problem-solving.
  - We will practice writing clean code using PEP (Python Enhancement Proposals).
  - We will also write codes on cloud computing resources.

About Computation for Analytics (MSDS 501)

Class Schedule

- This class will be offered in a flipped class - you will watch the pre-recorded sessions and join the following practice sessions.
  - Monday 10:00 - 11:50 AM PST (Room #154-156) - Moderate Level
  - Friday 10:00 - 11:50 AM PST (Room #154-156) - **Moderate + Hard Level (Mandatory)**

Office Hour

- Monday/Friday : 12 - 1pm (Room #154-156)

All student laptops must be closed during class unless we are doing a lab or I specifically ask you to follow along as I type into my computer. All materials for the course are available on Canvas and Github.

No cellphones, social media, slack, or texting during the class.

About Computation for Analytics (MSDS501)

Class Overview

Day 1
Class Intro, Basic Computer Architecture,
Terminal and Shell Commands

Day 2
Version Control & Git, Programming & Python,
Data Science Packages/Libraries, Debugging, Error Handling

Day 3
Python Data Types, Conditional Statements, Loop, pytest

Day 4
Python File I/O, Data Aliasing, Function

Day 5
Programming Patterns

Day 6
Packages, Libraries and Modules

Day 7
numpy

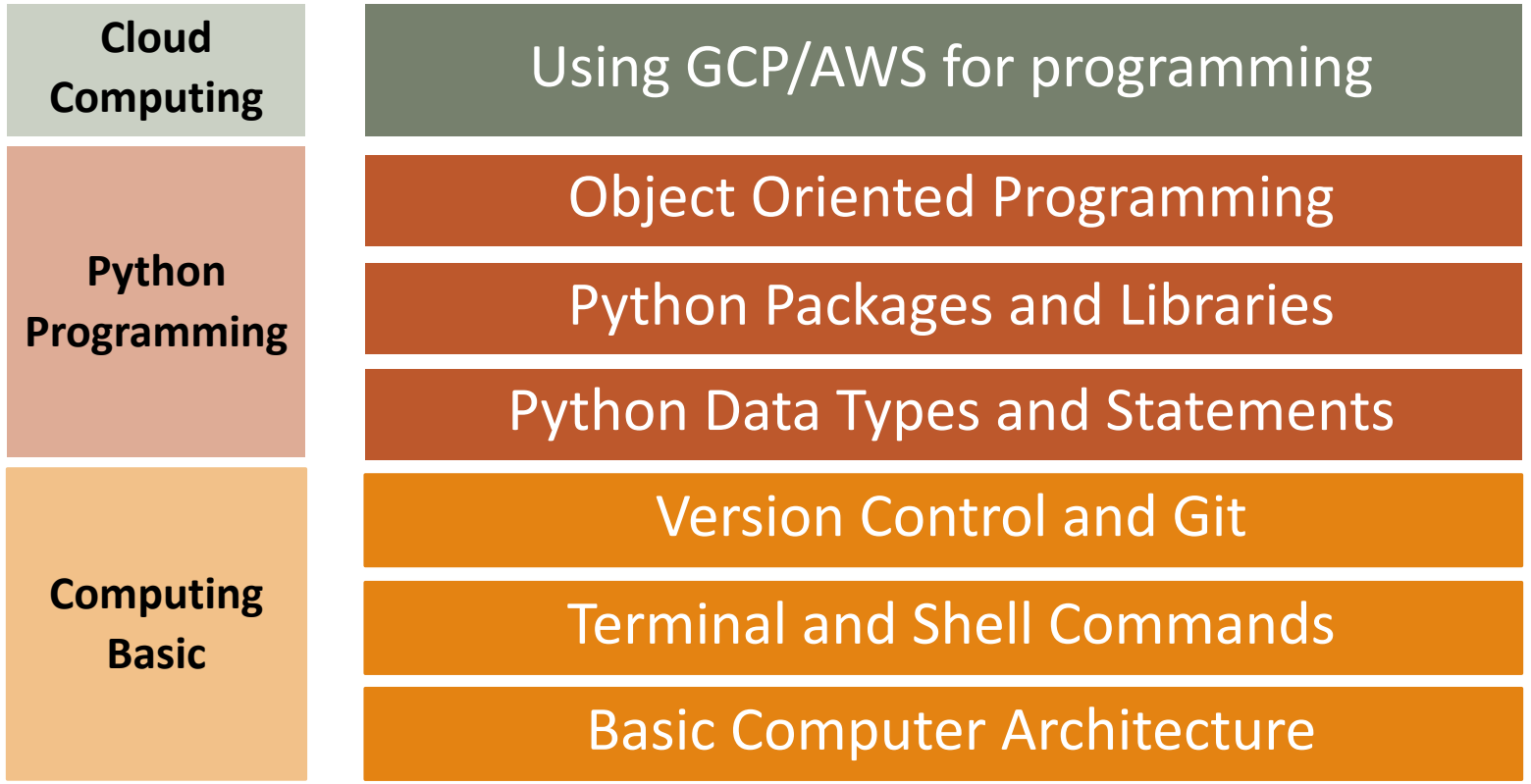
Day 8
Object Oriented Programming

Day 9
PEP8 and PEP20, Problem Solving

Day 10
Programming in Cloud Computing

Class Summary

Class Overview



For the students who are experts in programming



I have not yet seen students who are masters in programming.

- This is an introductory course to make everyone on the same page, and I expect you to master all the materials. Please review the materials thoroughly, as doing well in the boot camp courses are essential for proceeding to the program and being placed in your dream practicum.
- However, if you need more advanced materials, please get a Premium Leetcode account and start practicing.
  - Start with the Learn section and move forward.

About Computation for Analytics (MSDS501)

Evaluation Criteria

- Attendance and Professionalism - 10 %
- Weekly Assignments - 20 % (Due: 9pm on Thursdays)
- Quiz - 70 % (1-2pm on Fridays : Must be IN-PERSON, Remotely taken quiz will receive 0.)

Grader : To be announced

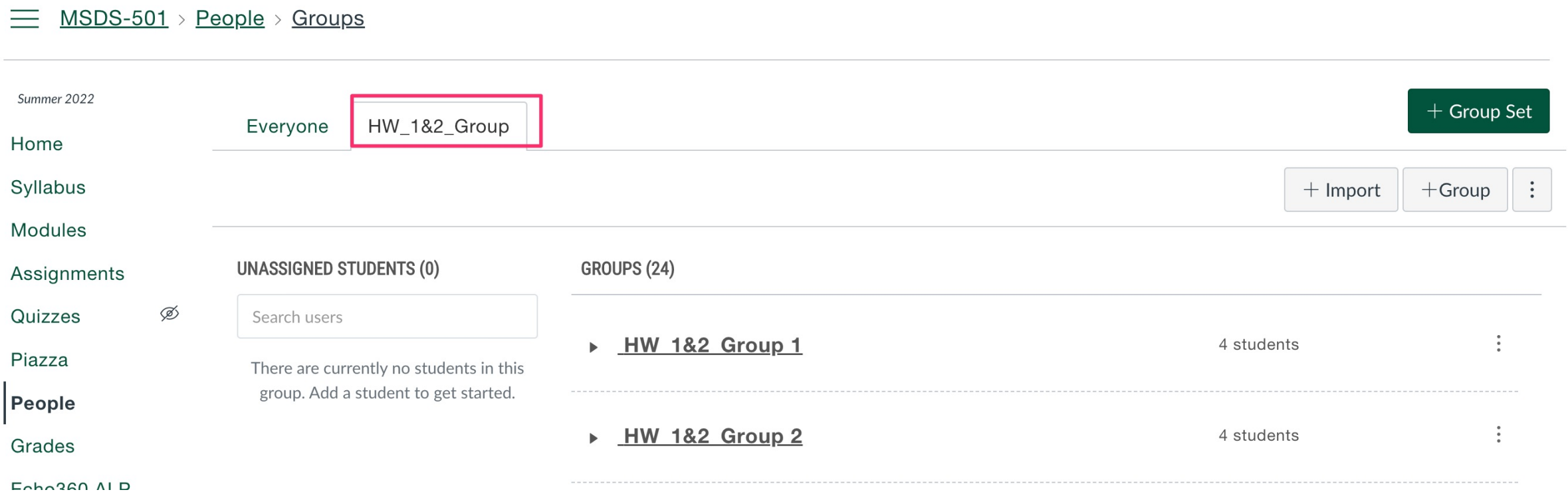
Weekly Assignment

All assignment for this course will be “Group Assignment” and you will have a group for HW_1_2 and HW_3_4_5.

- I will assign up to 3 ppl for group and you will submit one assignment per group. (Not every member needs to submit)
  - You can consider it as your study group.
  - For building networks, I will randomly assign groups.
- If you do not work as a group, please let me know today.
  - However, I found that working individually didn’t help my student who has a computer science degree - especially for building networks in the program.
- I am going to ask participation rate and your grade will be weighed based on it.
  - Tip : Make sure to do your homework as much as possible prior to your group meeting.

Weekly Assignment

It will be created on Canvas - Do not create your own group (it is not visible to me)



About Computation for Analytics (MSDS501)

This class is a standard, graded course with letter grades A - F.

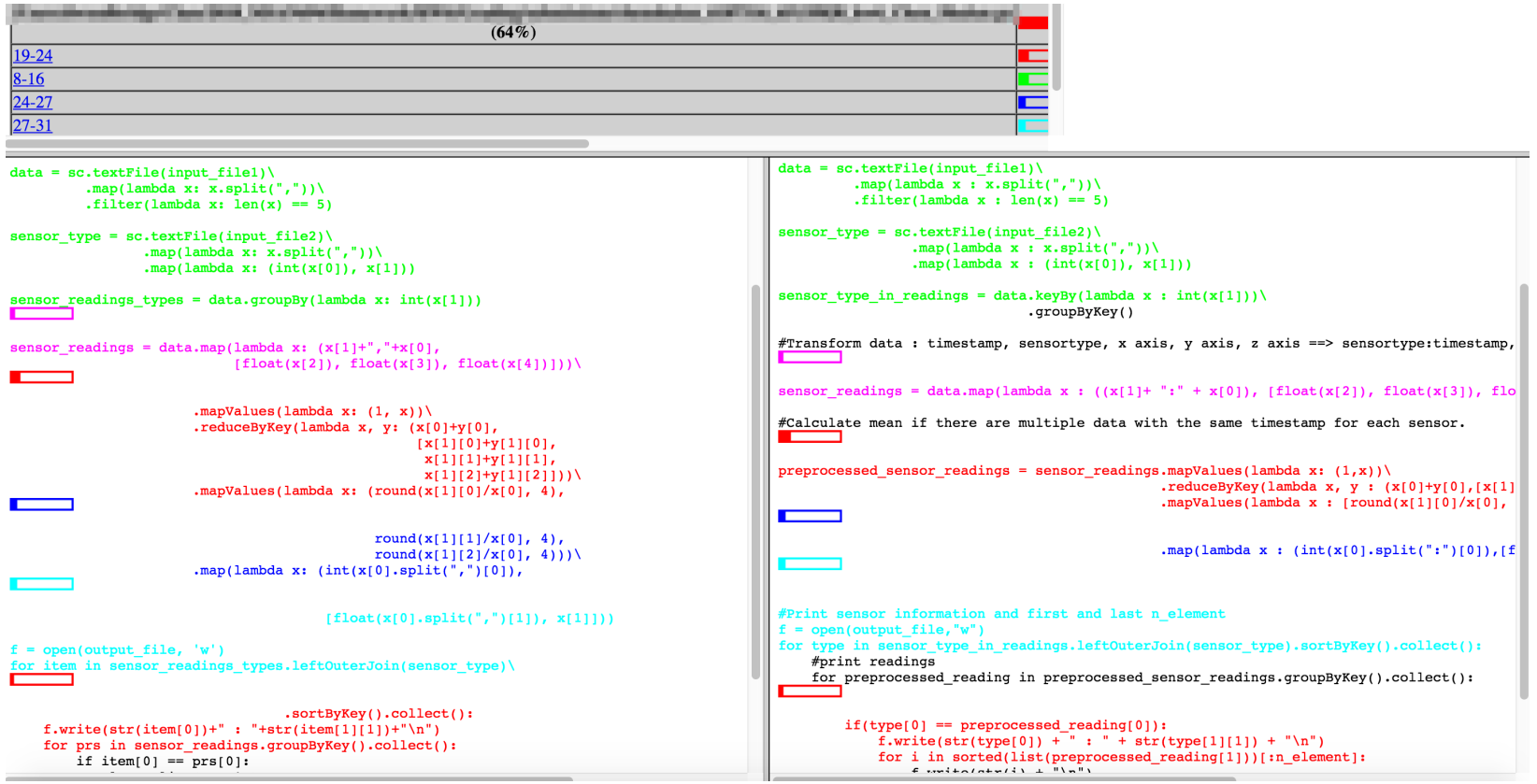
- I consider an A grade to be above and beyond what most students have achieved.
- A B grade is an average grade for a student or what you could call "competence" in a business setting.
- A C grade means that you either did not or could not put forth the effort to achieve competence.
- Below C (F) implies you did very little work or had great difficulty with the class compared to other students.

The expected final score for this course is 85 ± 3 (and close to normal distributions).
The following grades will be given if the class grade distribution falls within the expectation.
However, if the grade distribution does not meet the aforementioned criteria, grades will be curved.

Score	Letter Grades
90 - 100	A+, A and A-
80 - 90	B+, B and B-
70 - 80	C+, C and C-
Below 70	F

About Computation for Analytics (MSDS501)

Please no plagiarism! – Zero tolerant.



Student Engagement

Example Data : https://github.com/dianewoodbridge/msds501_computation_2023

Piazza : https://piazza.com/usfca/summer2023/msds501

Poll : https://pollev.com/msds

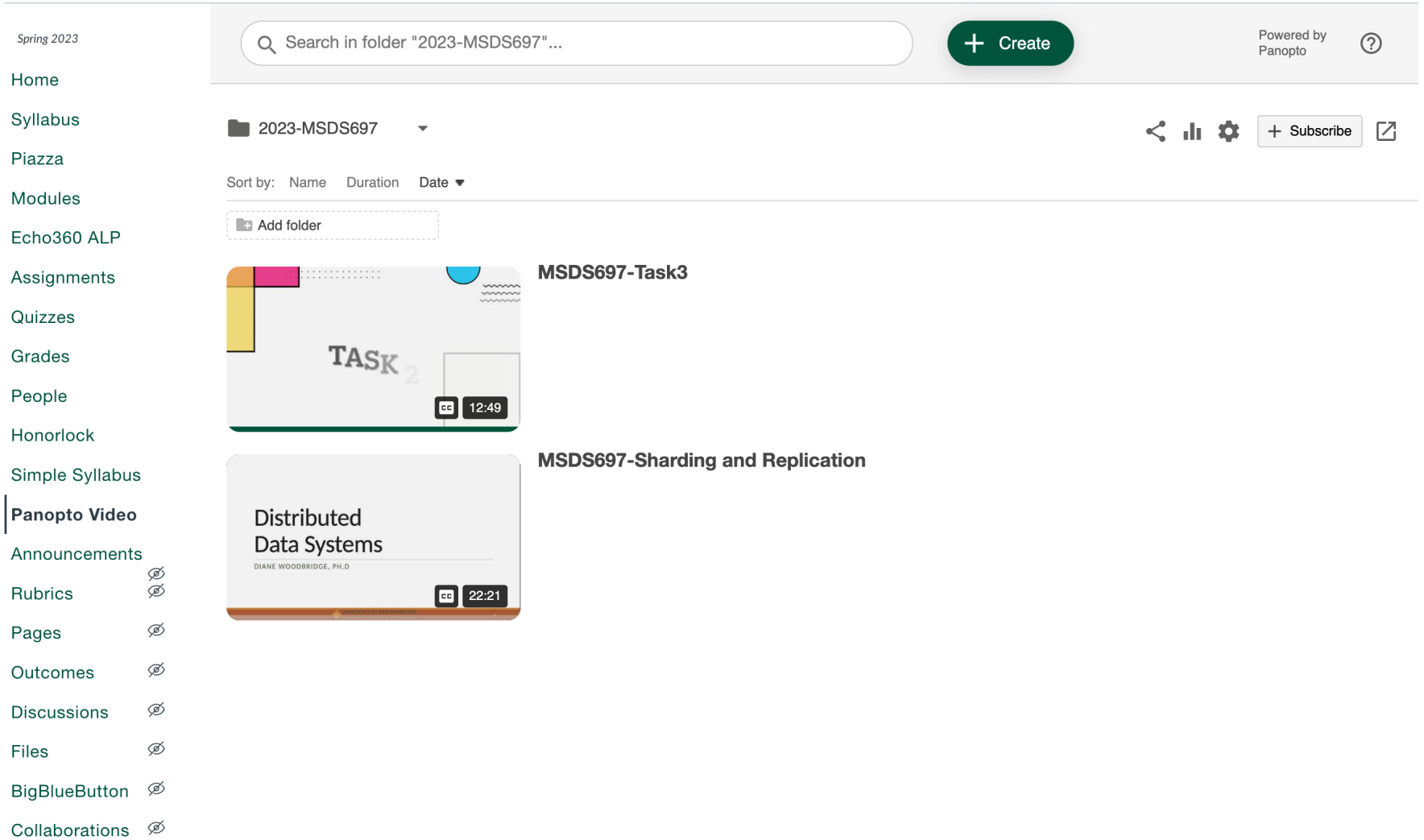
Canvas

- Under each module (week), there are learning outcome, slides, homework, example tests, tests, etc.
- Lectures will be recorded and uploaded on Panopto.
- Please Piazza for surveying topics for discussion sessions.

Student Engagement

Canvas

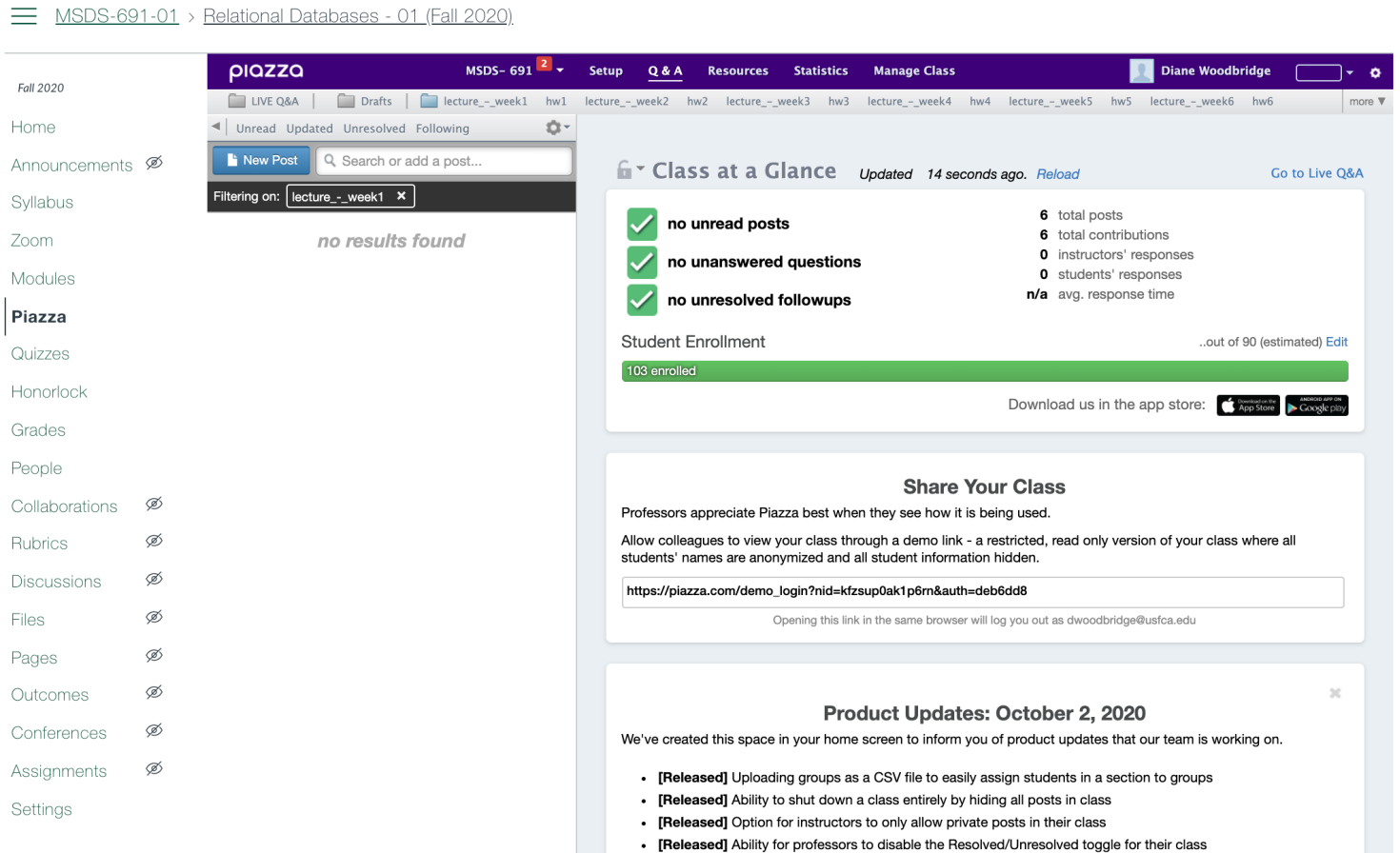
- Panopto



Student Engagement

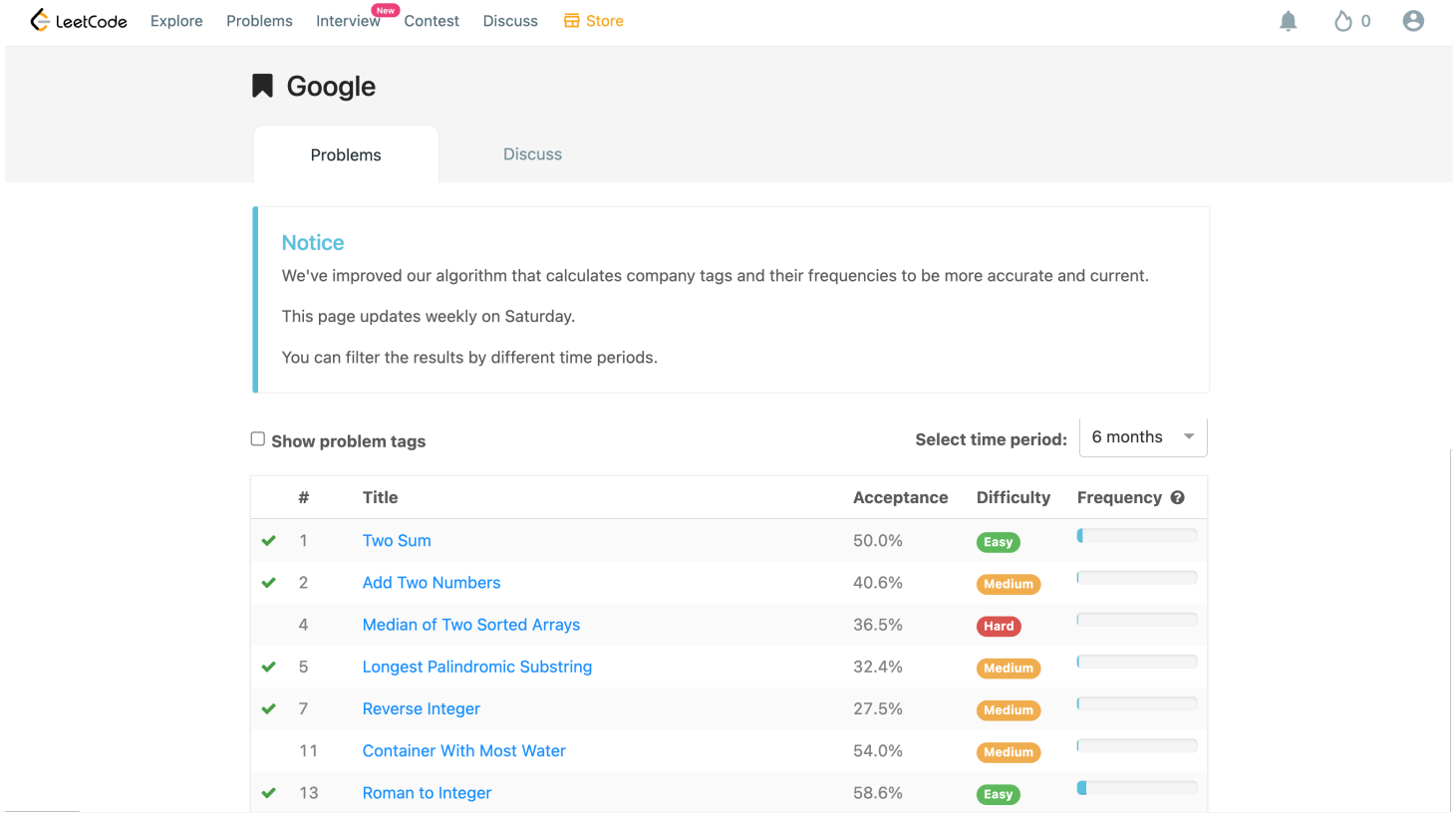
Canvas

- Piazza - For discussing lectures and assignments. (Preferable)



After this class...

I found that many of the students become comfortable to solve Easy - Medium level Leetcode questions.



Content

Basic Computer Architecture

Files and Folders

Terminal, Shell Commands/Scripts

Homebrew/Anaconda/Python Installation

Why did you purchase your computer??

Computer Programming Languages and Tools

Laptops

You'll be using your laptop every day, either in class or at Practicum. Your laptop must be in perfect working order and have the necessary computing power and storage. If you have a broken or slow laptop, it will directly affect your success in the program.

We strongly recommend you buy a Mac laptop with the new M1 processor, rather than the Intel processor. That means either the MacBook Air or MacBook Pro 13" which will have very similar performance. Get 16G not 8G of RAM, but the minimum 256G SSD storage is sufficient (minimum). The MacBook Air M1 with those specs is $1,199 without the educational discount (which is usually small). Student discounts on Apple products can be accessed at https://www.apple.com/us-hed/shop?aid=AIC-AOS-Edu-EdMktng-HP .

If you would like a bigger screen, you can look at the MacBook Pro 16" but you will pay a premium ($2,399). It has an Intel Core i7 processor that boosts up to 4.5 GHz and 16G RAM at that price. (December 23, 2020)

Content

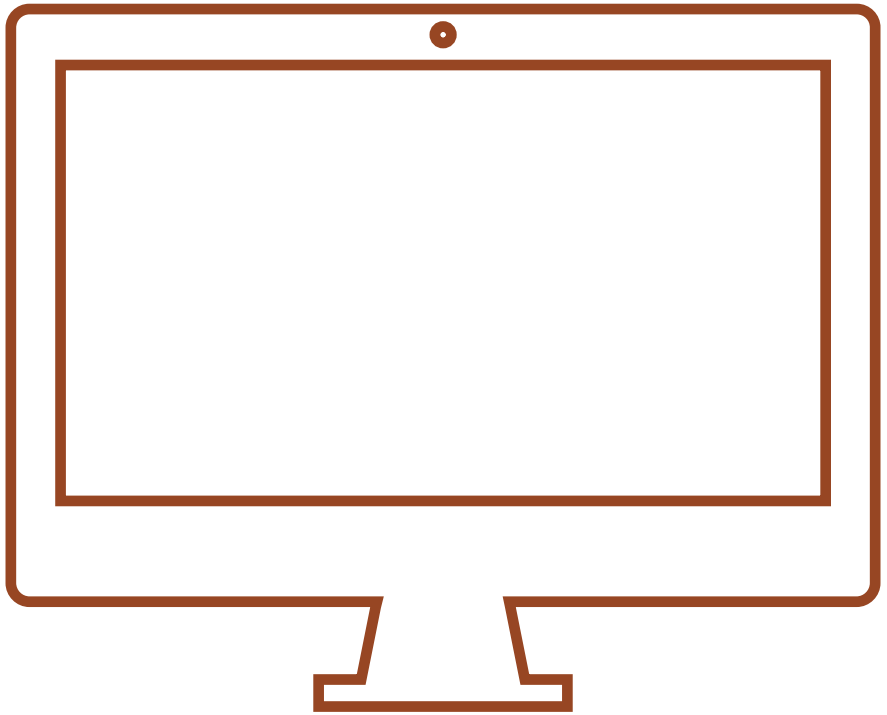
Basic Computer Architecture

Files and Folders

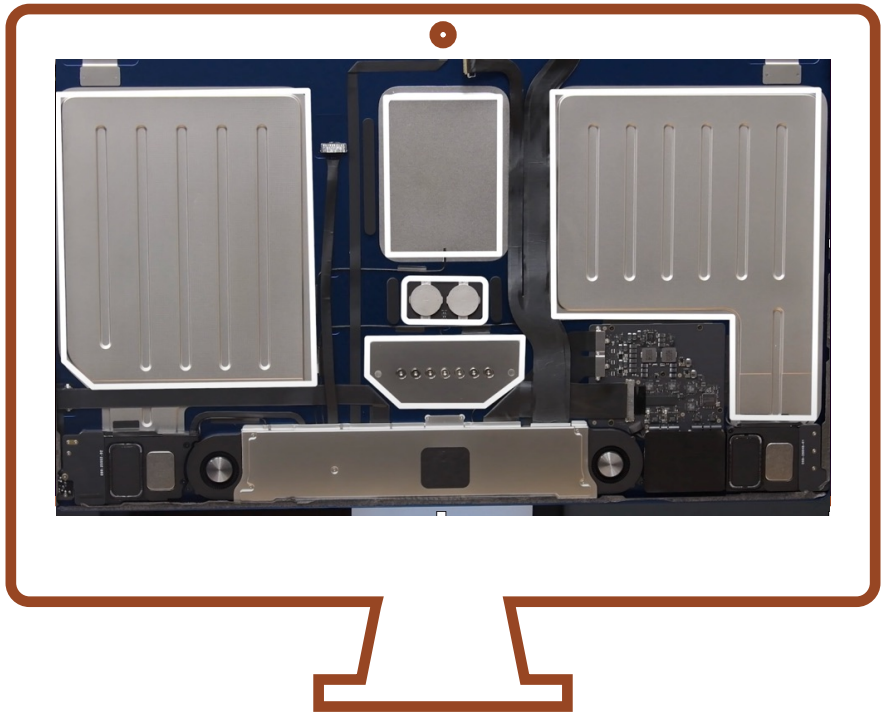
Terminal, Shell Commands/Scripts

Homebrew/Anaconda/Python Installation

Your Computer



Your Computer



Your Computer



youtube.com/watch?v=-WtGjkgIXGM&t=1s

Your Computer



Your Computer



CPU

- Core
- Thread



Memory (RAM)



Nonvolatile Storage (Disk)

Your Computer

CPU 

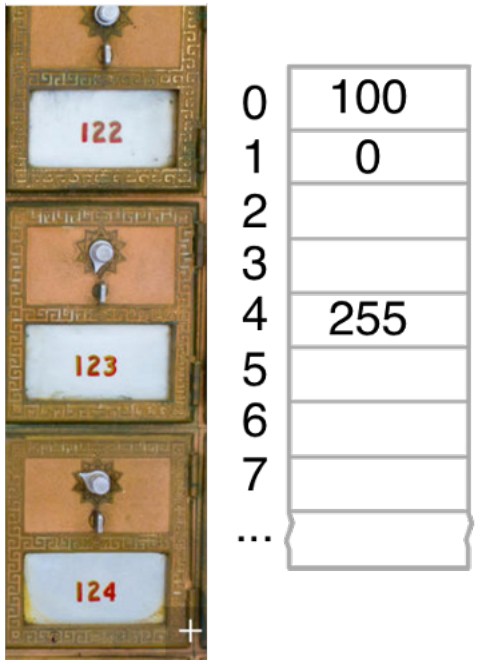
- Central Processing Unit, which executes operations/computations
- Performs five principal operations
  1. Load small chunks of data from memory into the CPU
  2. Perform arithmetic computations on data in the CPU
  3. Store small chunks of data back to memory
  4. Jump to a new location (this is how we loop)
  5. Jump to a new location if condition is true
- For CPU, there are few terminologies that you might want to remember.
  - Core : The number of workers that will process the job.
  - Thread : The number of work that each core can do concurrently. - You can think it as hands.

Your Computer

Memory (RAM)



- CPUs can read/write data at specific memory locations
- Disk vs RAM
  - RAM is much faster but usually much smaller than the disk and all RAM data is lost when the computer powers off
  - Think of memory as your working or scratch space and the disk as your permanent storage
- Memory is broken up into discrete cells of a fixed size called a byte that can hold a number between 0 and 255 (8 bits)
  - 1 Byte = 8 bit, 1 KB = 2^{10} Bytes = 1024 Bytes
  - Everything from actual numbers to music to videos is broken down and stored as a sequence of bytes

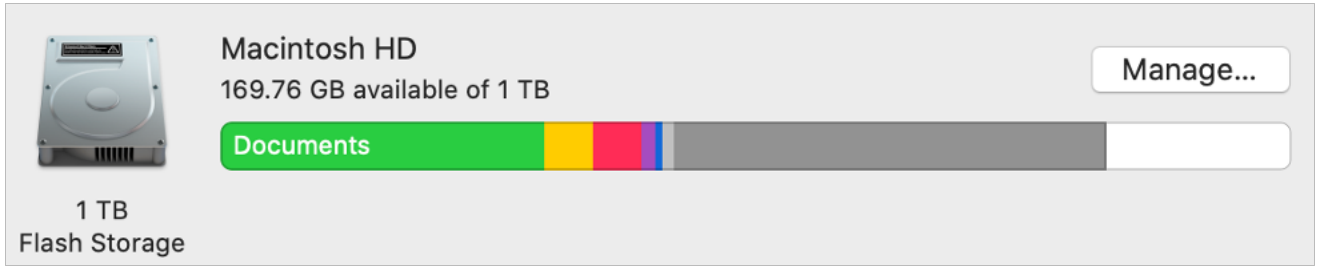


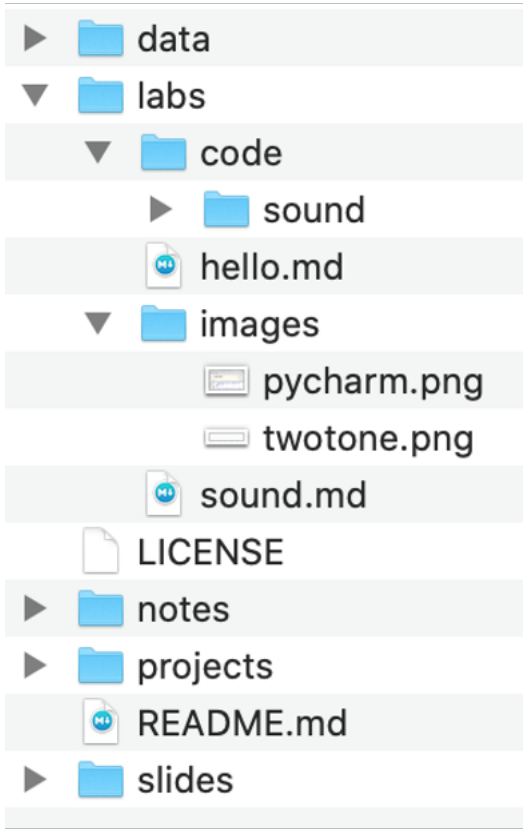
Your Computer

Nonvolatile Storage (Disk)

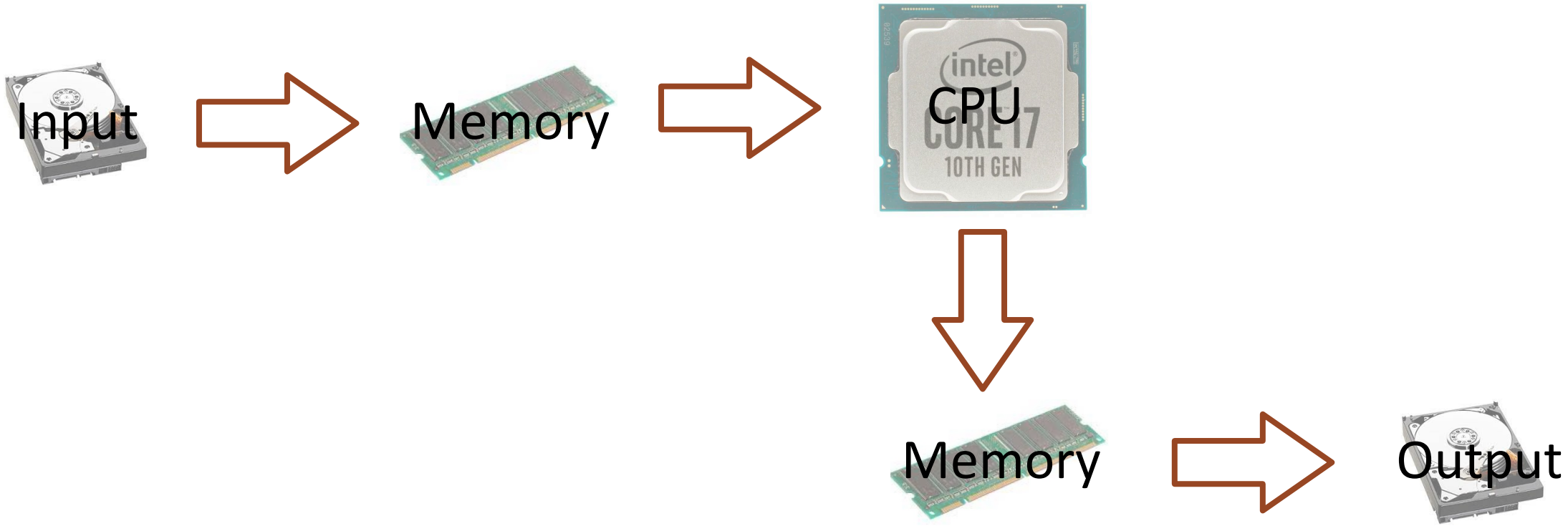


- Files hold data meant to be treated as a unit such as ladygaga.mp3 or sales.csv
- Directories (folders) group files and other(sub-)directories
- **Disk** is a tree of folders with files in the leaves





Read/process data from a file and write the results to a file.



Content

Basic Computer Architecture

Files and Folders

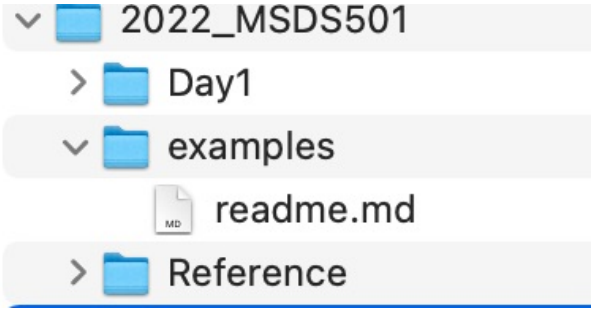
Terminal, Shell Commands/Scripts

Homebrew/Anaconda/Python Installation

Files and Folders

Executing programs have a current working directory and file specs (paths) relative to that.





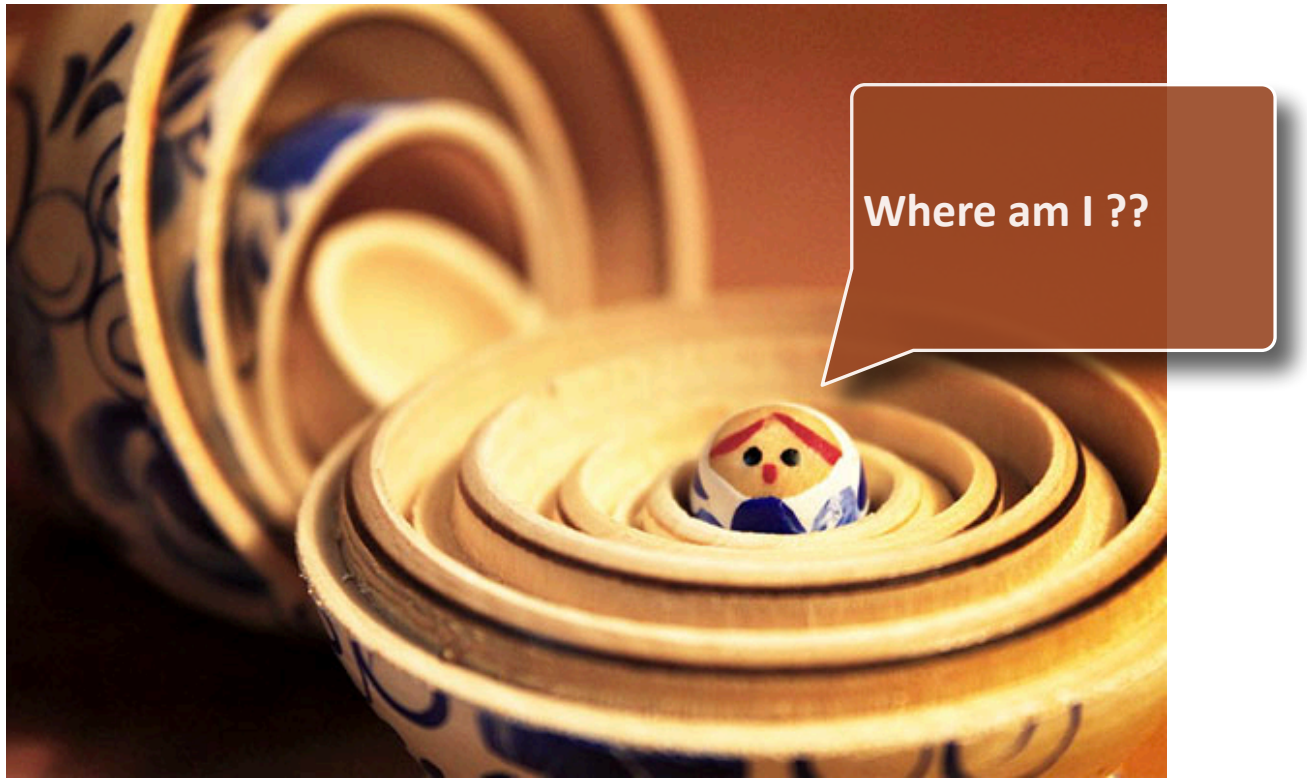
Files and Folders

Absolute Path and Relative Path

- **Absolute Path** : File or folder location (address) from the highest location (root directory)
  - Ex. `/Users/dwoodbridge/Class/2023_MSDS501/examples`
- **Relative Path** : File or folder location that is relative to a current directory
  - Current directory - where you are currently working
    - You can represent the current directory as `.`
  - Parent directory - A directory that contains the current directory
    - Ex. Parent directory of `/Users/dwoodbridge/Class/2023_MSDS501/examples` is /Users/dwoodbridge/Class/2023_MSDS501
    - You can represent parent directory as `../`

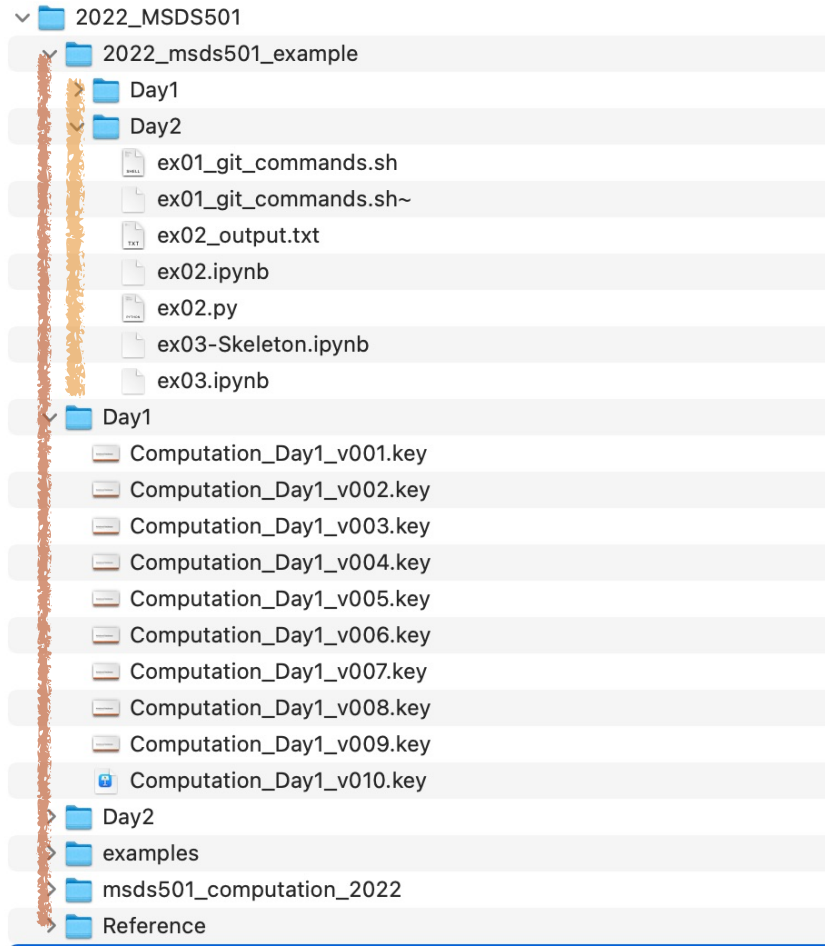
Files and Folders

Absolute Path and Relative Path



Example 1

For Day1/Computation_Day1_v001.key, what is the relative path for Day2/ex02.py?



Content

Basic Computer Architecture

Files and Folders

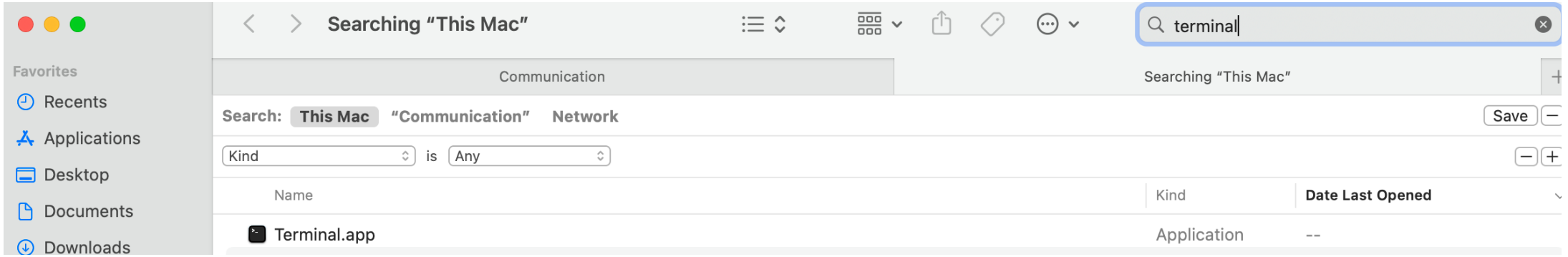
Terminal, Shell Commands/Scripts

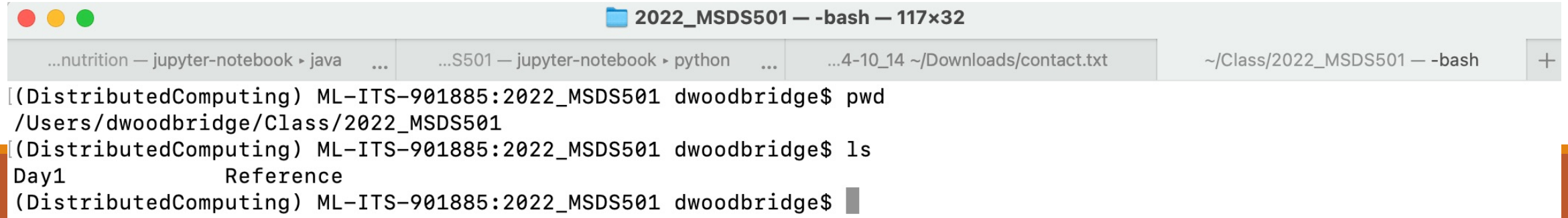
Homebrew/Anaconda/Python Installation

Terminal

Terminal provides a command line-interface (CLI) to MacOS.

- The terminal is running a "shell", a command interpreter designed to run programs, manage and interact with computers.
- In my class, I use $*commands* to distinguish terminal commands





Terminal 

Basic Terminal Commands

Type	Command	Explanation
Directory	`pwd`	Print current working directory
	`cd` *`path`*	Change current working directory
	`mkdir` *`name`*	Make a new directory
	`ls (-la)`	List files and subdirectories (with long format including hidden files)
	`rm -r` *`name`*	Remove/Delete the directory. **Carefully use it, as you cannot undo especially with -f option (force).**
File	`cp` *`from to`*	Copy “from” to “to”
	`mv` *`from to`*	Rename/Move “from” to “to”
	`cat` *`file`*	Print the content of the file.
	`touch` *`file`*	Create *file* without content.
	`vi/emacs` *`file`*	Open file using vi or emacs
	`diff` *`file_1 file_2`*	Return difference between file_1 and file_2
Permission	`chmod option` *`filename`*	Change the read, write, and execute permissions `option` - 3 digit for user, group, and others. Each digit is a sum of 4, 2, 1, or 0. (4: read, 2: write, 1: execute, and 0 : no permission)
Process	`ps aux`	Print all process of all users
	`kill` *`pid`*	kills the processes with pid

https://mally.stanford.edu/~sr/computing/basic-unix.html

Terminal 

Terminal Tips

- Tab does “auto-complete”
- If you are not specifying the directory, the commands work in your current working directory.
- If you get permission related errors, add `sudo` at the beginning of your commands.
- You can also pipeline a sequence of multiple commands by adding ' | ' or ' |& ‘
- Parameter -f is for “forcing”, -r is for “recursively”.
  Ex. If you need to delete all the files in a folder recursively, `$rm –r folder_name`

Example 2

- Create a folder called 2023_MSDS501_Day1 on the home (~) directory.
- In the current working directory, create *empty.txt*.
- Change your directory to 2023_MSDS501_Day1 and move *empty.txt* to here.
- Create an *empty_2.txt* with some contents.
- Compare *empty.txt* and *empty_2.txt*
- Change empty.txt to be readable/writeable and executable to everyone.
- Delete 2023_MSDS501_Day1 and all of its contents.

Terminal

Terminal Commands



Not familiar with shell commands and GUI seems waaaaay easy at this point?

Make no mouse/trackpad hours.
Within those hours, you are only allowed to use keyboards and shell commands.

Shell Scripts

A script including shell commands for directory/ file manipulation, program execution, etc.

- File extension : `.sh`
- You can execute it by `$sh xxx.sh`
- You can combine with other control sequences including for variable assignments, loop, conditional statements, etc. (Note : Those syntax are different from Python syntax)

Shell Scripts

Variable Assignment

- Use = (equal sign) without any space before/after. Ex. a=13
- To refer to the variable after assignment, you need to add $ before the variable name. Ex. echo $a

Loops

- Create a block of

```
for arg in list_of_items_separated_by_space
    do
        command(s)...
done
```

Conditional Statement

- Create a block of

```
if [ condition ]
    then command(s)
fi
```

Example 3

For numbers in ***nums*** defined in ex03.sh, print if the number is greater than ***threshold.***

Array Variables : https://tldp.org/LDP/Bash-Beginners-Guide/html/sect_10_02.html

Comparison Operator : https://tldp.org/LDP/abs/html/comparison-ops.html

Environment Variables

User defined variable that is used for running processes on your computer.

- You have to set it often as a data scientist such as using cloud computing, writing code without setting sensitive information explicitly (password), etc.

Setting Environment Variables

- You can set an environment variable by `$export ENV_NAME=VALUE`
  - Make sure not to use any space before/after =
- You can check whether it is correctly set by `$echo $ENV_NAME`
- Recommendation : If those are going to be repeatedly used, add to ~/.bash_profile.sh or ~/.zshrc
  - After updating it either open a new terminal or do `$source ~/.bash_profile` or `$source ~/.zshrc`

Content

Basic Computer Architecture

Files and Folders

Terminal, Shell Commands/Scripts

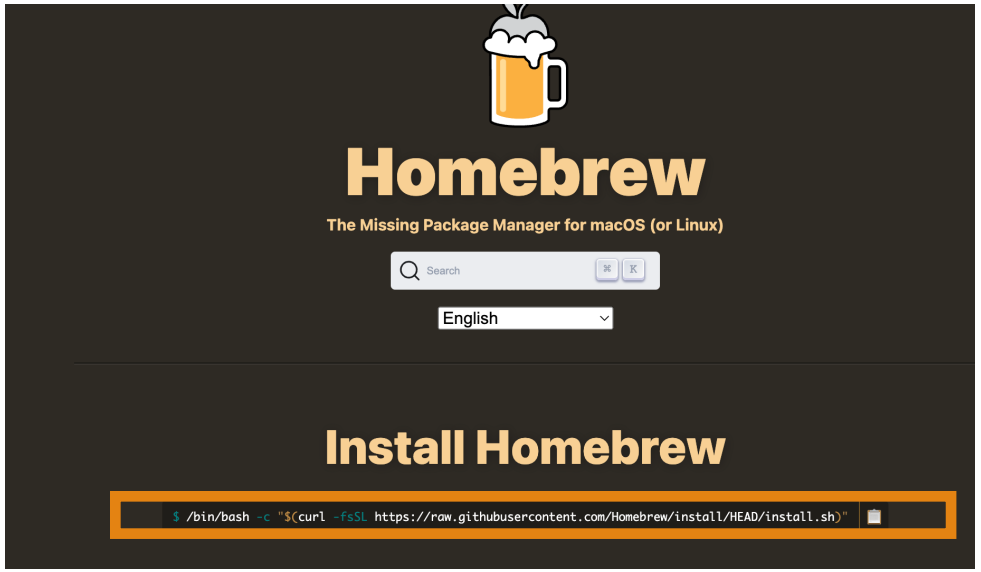
Homebrew/Anaconda/Python Installation

Let's Get Ready

Installation

- Homebrew
  - For installing missing packages for MacOS
  - Download and Install

```
$/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/
Homebrew/install/HEAD/install.sh)"
```



https://brew.sh/

Let's Get Ready

Installation

- Anaconda
  - Package management tool - It supports more than Python distributions
    - We will install packages using Anconda using `$conda install package_name`
      - Tips : Use `$conda install` first rather than `$pip install`
  - Download and Install

```
$ brew install --cask anaconda
```

https://formulae.brew.sh/cask/anaconda
https://docs.brew.sh/Formula-Cookbook#homebrew-terminology

Let's Get Ready

Installation

- Python - Install version 3.11
  - Using conda, it will install other packages including numpy, scipy, pandas, jupyter. etc.
  - To install a certain version of python
    `$conda install python=3.x`

```
(base) dwoodbridge@ML-ITS-210588 ~ % conda install python=3.11
Retrieving notices: ...working... done
Collecting package metadata (current_repodata.json): done
Solving environment: \
```

https://docs.anaconda.com/anaconda/install/mac-os/

Content

Basic Computer Architecture

Files and Folders

Terminal, Shell Commands/Scripts

Homebrew/Anaconda/Python Installation

References

M1 iMac Teardown : youtube.com/watch?v=-WtGjkgIXGM&t=1s

Advanced Bash-Scripting Guide : https://tldp.org/LDP/abs/html/

Homebrew : https://brew.sh/

Anaconda : https://docs.anaconda.com/

Shell Commands : https://mally.stanford.edu/~sr/computing/basic-unix.html